CYCLE - 1

EXPERIMENT - 1

AIM : Familiarization to computer hardware such as CPU, Memory, Motherboard etc .

Theory:

A computer system consists of several hardware components, each playing a critical role in its operation.

The main components include:

1- CPU (Central Processing Unit)

The CPU is the most important component of a computer system and is often referred to as the *brain of the computer*. It interprets and executes instructions from software programs, performs arithmetic and logical operations, and controls the overall functioning of the system. The CPU consists of the **Arithmetic Logic Unit (ALU)**, **Control Unit (CU)**, and **registers**. Its performance depends on clock speed, number of cores, and cache memory.

- Speed measured in **GHz**
- Has multiple **cores** (dual-core, quad-core, etc.)
- Examples: Intel Core i5/i7, AMD Ryzen

2. Memory (RAM – Random Access Memory)

RAM is the primary memory of the computer used to temporarily store data and instructions that are currently being processed by the CPU. It allows fast access to information, which helps the system run efficiently. RAM is volatile in nature, meaning all stored data is lost when the computer is powered off. Larger RAM capacity improves multitasking and system responsiveness.

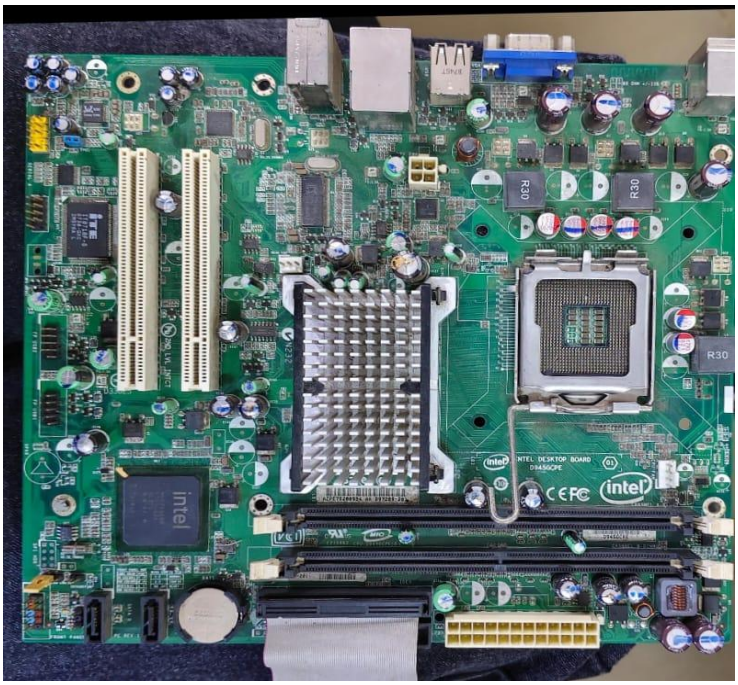
- Measured in **GB** (8GB, 16GB, etc.)
- Data is lost when power is off (volatile)

- More RAM = smoother performance

3. Motherboard

The motherboard is the main circuit board that holds and connects all internal components of the computer. It provides sockets for the CPU, slots for memory modules, connectors for storage devices, and expansion slots for additional hardware like graphics and sound cards. It also contains chipsets that manage data flow between the processor, memory, and peripherals.

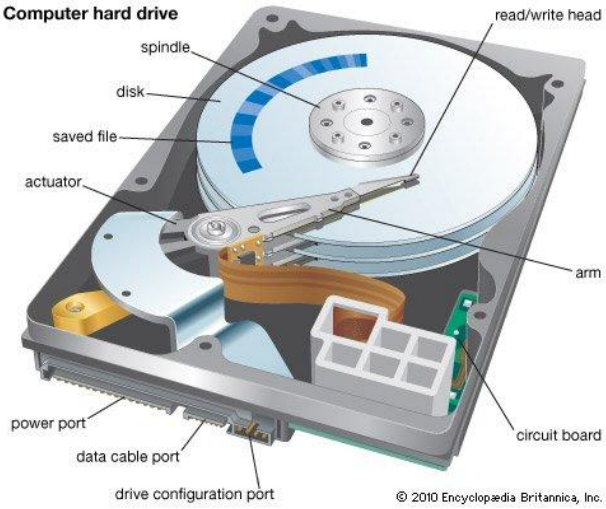
- Determines what CPU and RAM types you can use
- Contains slots, ports, and connectors
- Sizes: ATX, Micro-ATX, Mini-ITX



4. Storage Devices

a) Hard Disk Drive (HDD)

The hard disk drive is a secondary storage device used for permanent data storage. It stores the operating system, software applications, and user files. HDDs use magnetic disks to store data and provide large storage capacity at a low cost, but they are slower compared to modern storage devices.



b) Solid State Drive (SSD)

An SSD is a high-speed storage device that uses flash memory to store data. Unlike HDDs, SSDs have no moving parts, making them faster, more reliable, and energy-efficient. SSDs significantly reduce system boot time and application loading time, improving overall performance.

5. Power Supply Unit (PSU)

The power supply unit converts high-voltage alternating current (AC) from the power outlet into low-voltage direct current (DC) required by computer components. It distributes power to the motherboard, storage devices, and other peripherals while protecting them from power surges and electrical damage.

- Converts AC to DC power
- Measured in **Watts** (450W, 650W, etc.)
- Quality PSU protects components

6. Input Devices

Input devices allow users to enter data, commands, and instructions into the computer system. These devices convert user actions into signals that the computer can process. Common input devices include the keyboard, mouse, scanner, joystick, and microphone.

Examples:

- Keyboard
- Mouse
- Scanner
- Webcam

7. Output Devices

Output devices present the results of computer processing in a human-readable form. They convert digital signals into visual, audio, or printed output.

Examples:

- Monitor
- Printer
- Speakers
- Headphones

8. Graphics Card (GPU)

The GPU is a specialized processor designed to handle graphics and visual computations. It accelerates image rendering, video playback, and 3D graphics. GPUs can be integrated into the CPU or available as dedicated expansion cards, and they are essential for tasks such as gaming, video editing, and graphic design.

- Important for gaming, video editing, AI
- Can be **integrated** or **dedicated**
- Examples: NVIDIA, AMD Radeon

9. Cooling System

The cooling system helps maintain safe operating temperatures inside the computer. Components like the CPU and GPU generate heat during operation, and cooling devices such as heat sinks, cooling fans, and liquid cooling systems prevent overheating and hardware damage.

- Provides airflow and protection
- Sizes match motherboard type

EXPERIMENT -2

AIM : Hardware specifications and features of the latest AI - powered desktop and server computers available in 2025.

Theory

AI-powered computers require specialized hardware components to process large datasets and perform complex computations efficiently. These systems are designed with powerful processors, GPUs, large memory capacity, and fast storage to support machine learning and deep learning tasks.

The main hardware components include:

1. High-Core CPUs

Modern AI computers use multi-core processors such as Intel Xeon or AMD Ryzen processors. These CPUs handle system operations, data preprocessing, and task coordination. High core counts allow parallel processing of multiple tasks simultaneously.

2. GPUs with CUDA / Tensor Cores

Graphics Processing Units (GPUs) accelerate AI computations because they can perform thousands of parallel calculations. GPUs such as NVIDIA A100, H100, or RTX series contain CUDA and Tensor cores that are optimized for deep learning algorithms.

3. High-Capacity RAM

Large memory capacity is required to store datasets, neural network parameters, and intermediate computations. AI systems often include 64 GB to several terabytes of RAM depending on the workload.

4. NVMe SSD Storage

AI workloads require high-speed data access. NVMe SSDs provide faster read/write speeds compared to traditional HDDs or SATA SSDs, improving training and data processing speed.

5. AI Accelerators

Some modern systems also include Neural Processing Units (NPUs) or AI accelerators that further optimize AI model inference and training.

1Comparison Chart of AI-Powered Hardware (2025)

Hardware Component	AI Desktop System	AI Workstation	AI Server / Data Center
CPU	Intel Core Ultra / Ryzen 9 (16–24 cores)	Threadripper Pro / Xeon W (24–64 cores)	AMD EPYC / Xeon Platinum (100+ cores)
GPU	NVIDIA RTX 4070 / 4080	RTX 6000 Ada / RTX 5090	NVIDIA H200 / B200
GPU Memory	12–16 GB GDDR6/7	32–96 GB VRAM	Up to 141–192 GB HBM3e per GPU
System RAM	32–64 GB DDR5	64–512 GB DDR5 ECC	512 GB – 4 TB DDR5
Storage	1–2 TB NVMe SSD	2–8 TB NVMe SSD	10–100 TB NVMe storage
Purpose	Local AI experiments	Professional model training	Large-scale AI and LLM training

How These Specifications Enhance AI Workloads

1. **Parallel Processing**
   Multi-core CPUs and GPUs allow thousands of calculations to run simultaneously, speeding up machine learning training.
2. **Faster Model Training**
   GPUs with Tensor Cores accelerate matrix operations used in neural networks.
3. **Handling Large Datasets**
   Large RAM capacity allows systems to process massive datasets used in AI models.
4. **High-Speed Data Access**
   NVMe SSD storage reduces data loading time during training.
5. **Scalability for Large Models**
   AI servers combine multiple GPUs using high-speed interconnects like NVLink, enabling training of extremely large models.

EXPERIMENT – 3

AIM : Installation of linux on virtual box in windows 10

Procedure :

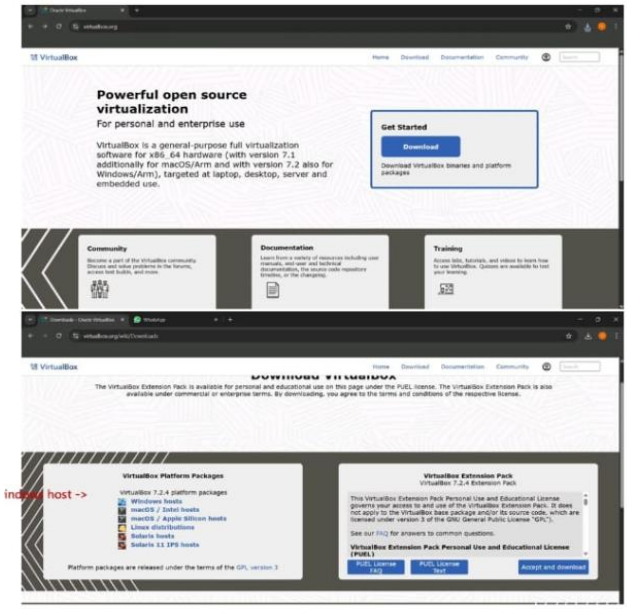
Step 1: Check System Requirements

Ensure your Windows computer supports virtualization.

- 64-bit processor
- At least 4 GB RAM (8 GB recommended)
- Virtualization enabled in BIOS/UEFI

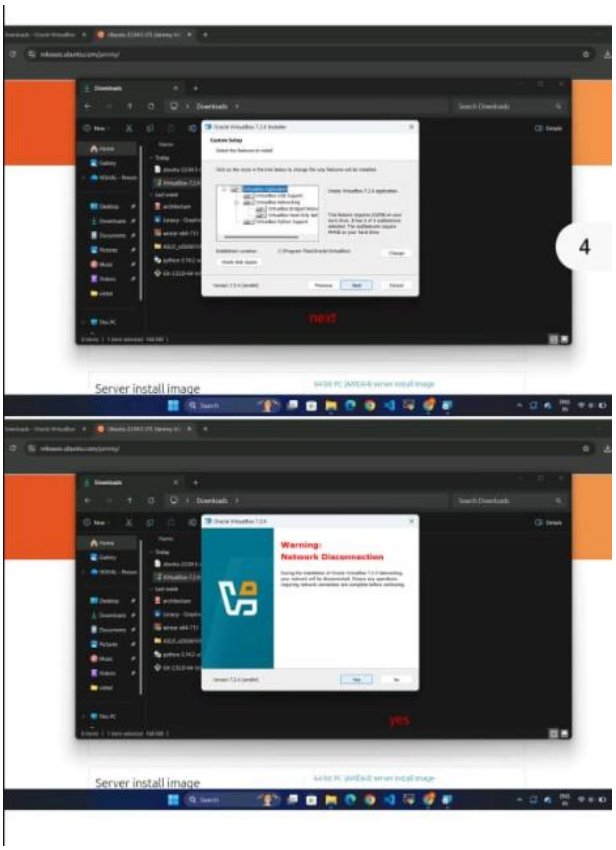
Step 2: Download VirtualBox

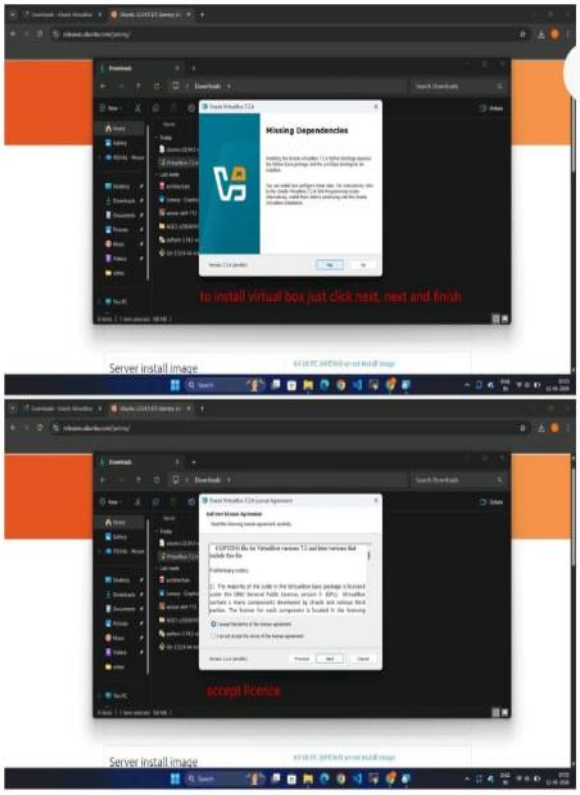
1. Open a web browser on Windows.
2. Go to the official VirtualBox website: **www.virtualbox.org**
3. Click on **Downloads**.
4. Select **Windows Hosts** to download the VirtualBox installer.
5. Save the installer file on your computer.



Step 3: Install VirtualBox

1. Double-click the downloaded VirtualBox installer.
2. Click **Next** in the setup wizard.
3. Choose installation options (default settings are recommended).
4. Click **Install** and allow permissions when prompted.
5. Wait for installation to complete, then click **Finish**.





Step 4: Choose a Linux Distribution

Select a Linux operating system such as:

- Ubuntu
- Fedora
- Linux Mint

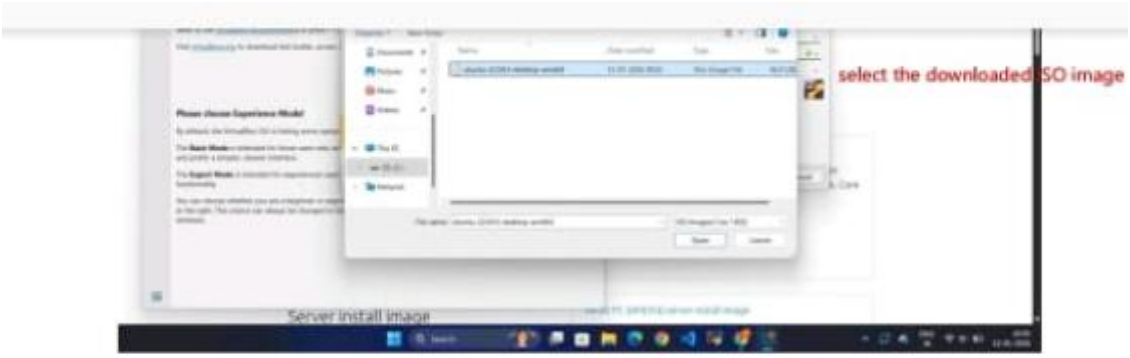
Step 5: Download Linux ISO

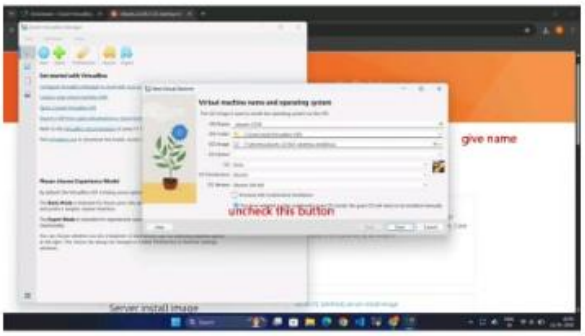
1. Visit the official website of the chosen Linux distribution.
2. Download the **ISO file** (disk image).
3. Save the ISO file to your system.

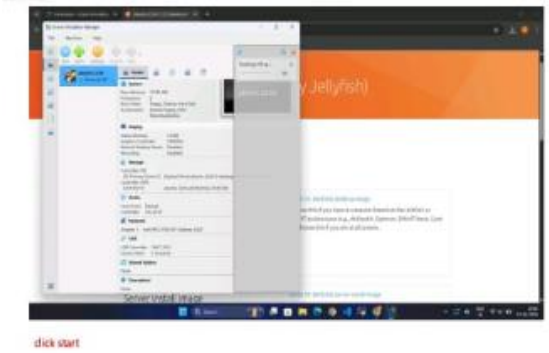
Steps to Install Linux on VirtualBox

Step 6: Create a New Virtual Machine

1. Open **VirtualBox**.
2. Click **New**.
3. Enter the name of the Linux OS (e.g., Ubuntu).
4. Select **Type: Linux** and choose the correct **Version**.
5. Click **Next**.







Step 7: Allocate Memory and CPU

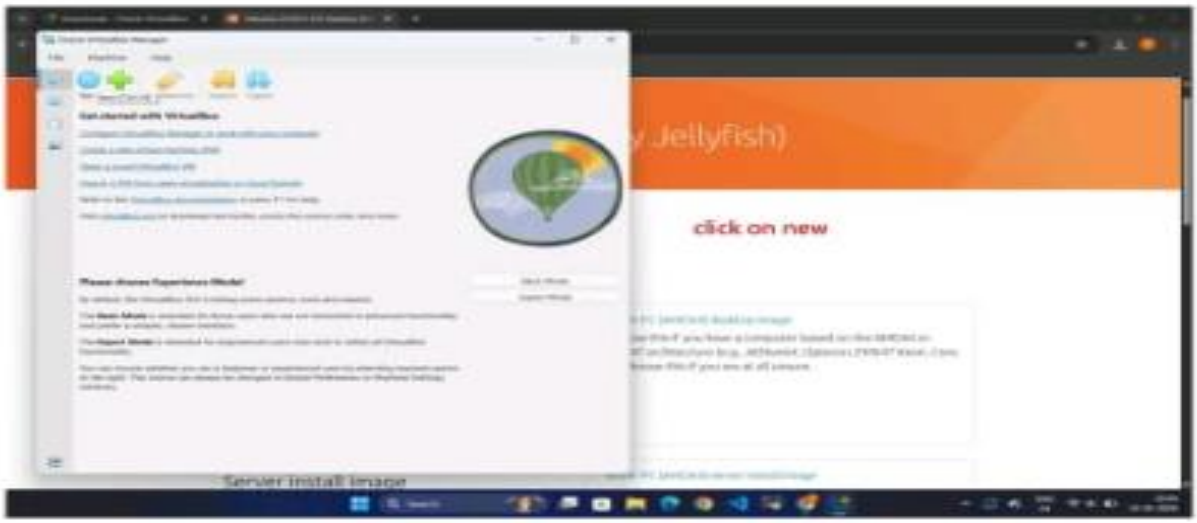
1. Assign RAM (minimum 2 GB recommended).
2. Allocate CPU cores if prompted.
3. Click **Next**.

Step 8: Create Virtual Hard Disk

1. Choose **Create a virtual hard disk now**.
2. Select **VDI (VirtualBox Disk Image)**.
3. Choose **Dynamically allocated**.
4. Set disk size (minimum 20 GB recommended).
5. Click **Create**.

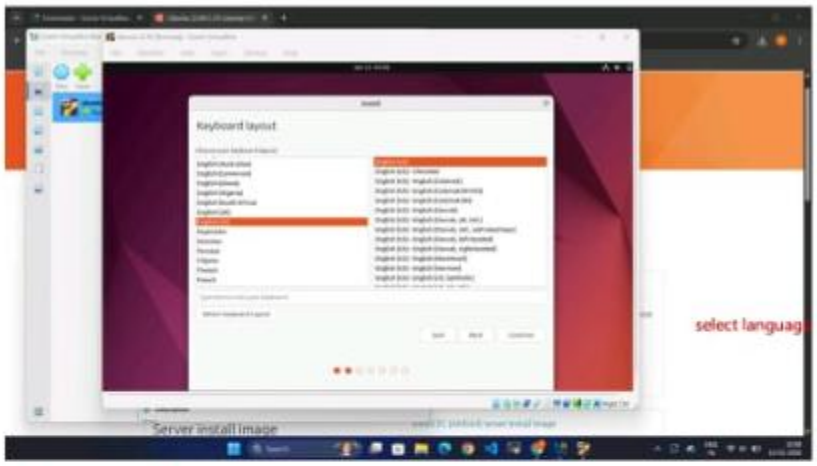
Step 9: Attach Linux ISO File

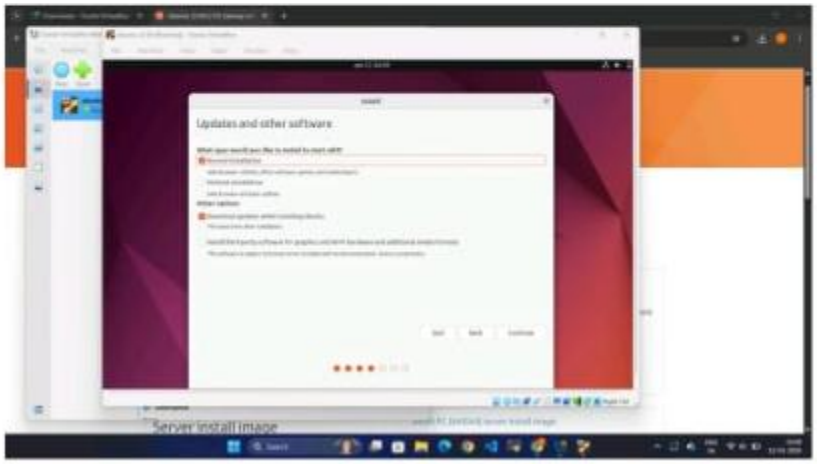
1. Select the created virtual machine.
2. Click **Settings → Storage**.
3. Under Controller IDE, click **Empty**.
4. Choose **Add Disk → Choose a disk file**.
5. Select the downloaded Linux ISO file.
6. Click **OK**.



Step 10: Start Linux Installation

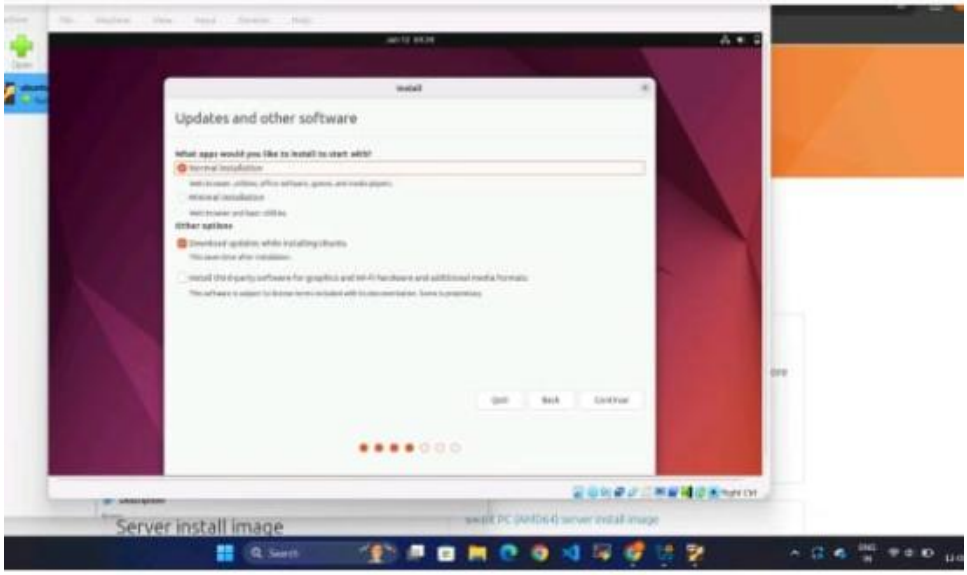
1. Click **Start** in VirtualBox.
2. The Linux installer will boot from the ISO file.
3. Choose **Install Linux**.
4. Select language, keyboard layout, and region.
5. Choose **Erase disk and install Linux** (affects only the virtual disk).
6. Create user account and password.
7. Click **Install Now**.

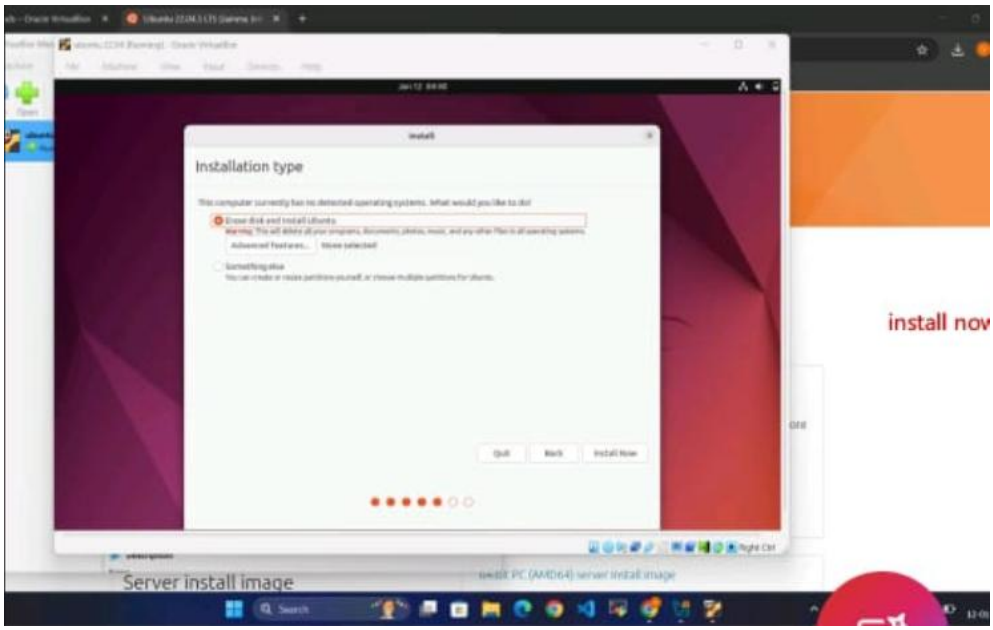




Step 11: Complete Installation

1. Wait for installation to finish.
2. Restart the virtual machine when prompted.
3. Remove the ISO file if asked.





Conclusion

VirtualBox allows Linux to run inside Windows without affecting the main system. Installing Linux in VirtualBox is safe, flexible, and useful for learning, testing, and development.

EXPERIMENT – 4

AIM : Installation of windows on virtual box in Ubuntu

Procedure :

Step 1: Install VirtualBox on Ubuntu

Open Terminal and run:

- sudo apt update

- sudo apt install virtualbox

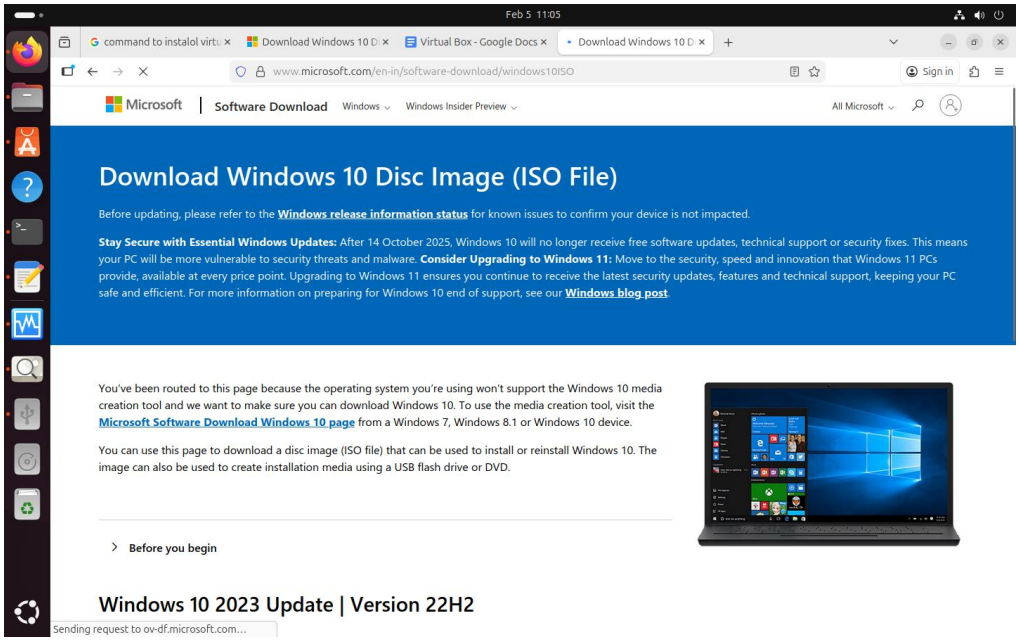
- sudo apt install dkms

```
mca87@mca87-HP-Pavilion-Desktop-590-p0xxx:~$ sudo apt install virtualbox
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 8113 (unattended-upgr)
```

```
mca87@mca87-HP-Pavilion-Desktop-590-p0xxx:~$ sudo apt install dkms
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
dkms is already the newest version (3.0.11-1ubuntu13).
0 upgraded, 0 newly installed, 0 to remove and 233 not upgraded.
```

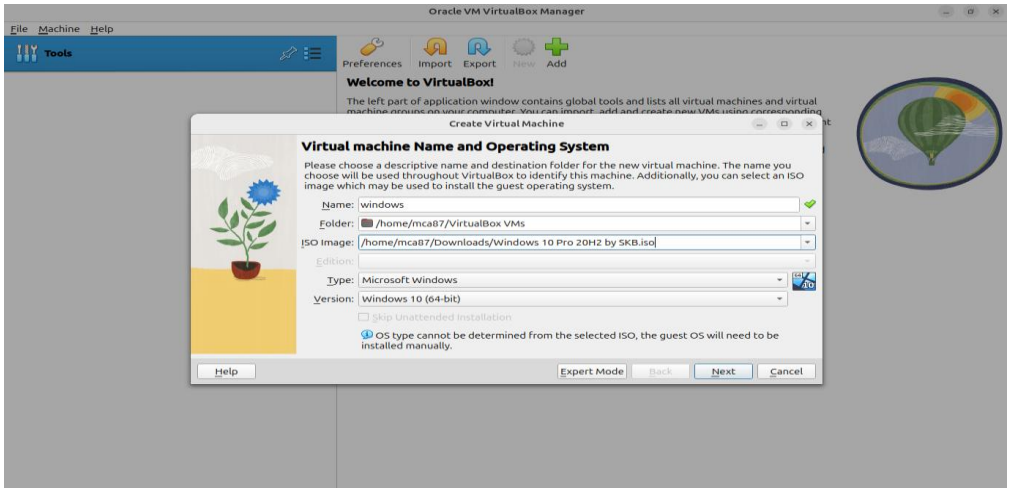
Step 2: Download Windows ISO

1. Windows 10 ISO: from Microsoft's official site
2. Windows 11 ISO: also from Microsoft



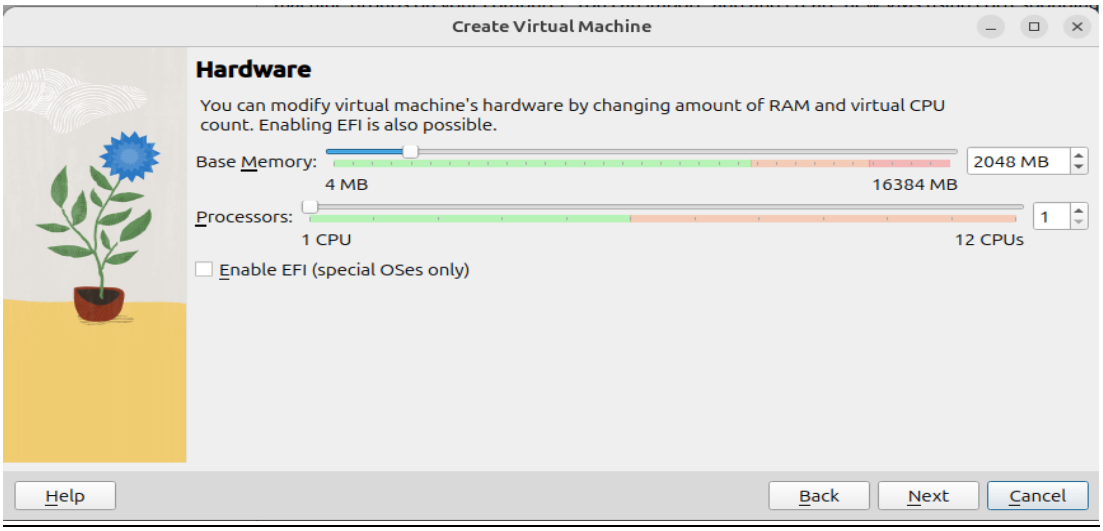
Step 3: Create a New Virtual Machine

1. Open **Virtual Box**
2. Click **New**
3. Set:
4. **Name:** Windows 10 / Windows 11
5. **Type:** Microsoft Windows
6. **Version:** Windows 10 (64-bit) or Windows 11 (64-bit)



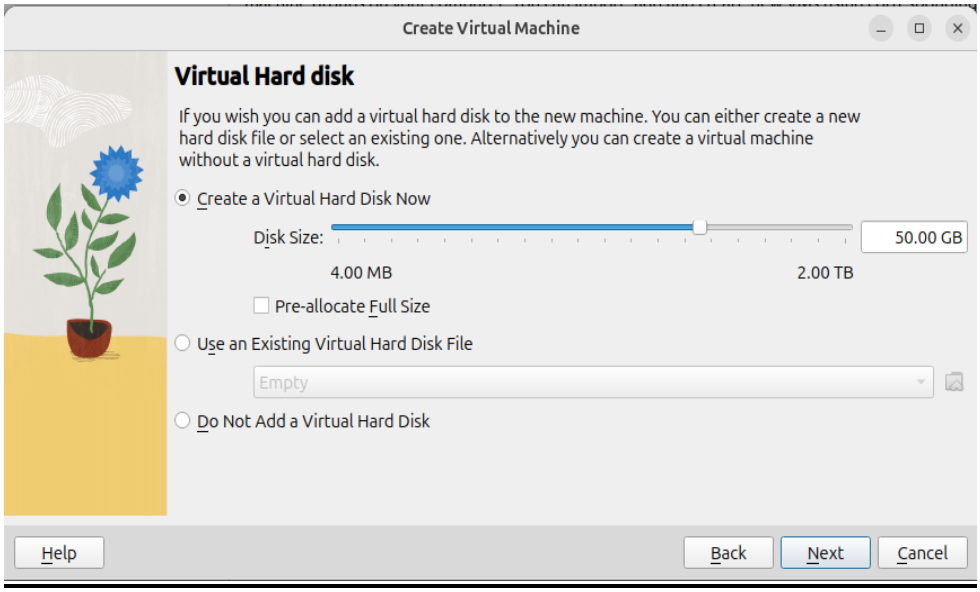
Step 4: Allocate Memory (RAM)

1. Minimum: **2048 MB (4 GB)**
2. Recommended: **8192 MB (8 GB)** if your system allows



Step 5: Create Virtual Hard Disk

1. Choose **Create a virtual hard disk now**
2. Disk type: **VDI**
3. Storage: **Dynamically allocated**
4. Size:
    a. Windows 10: **50 GB**
    b. Windows 11: **64 GB+**



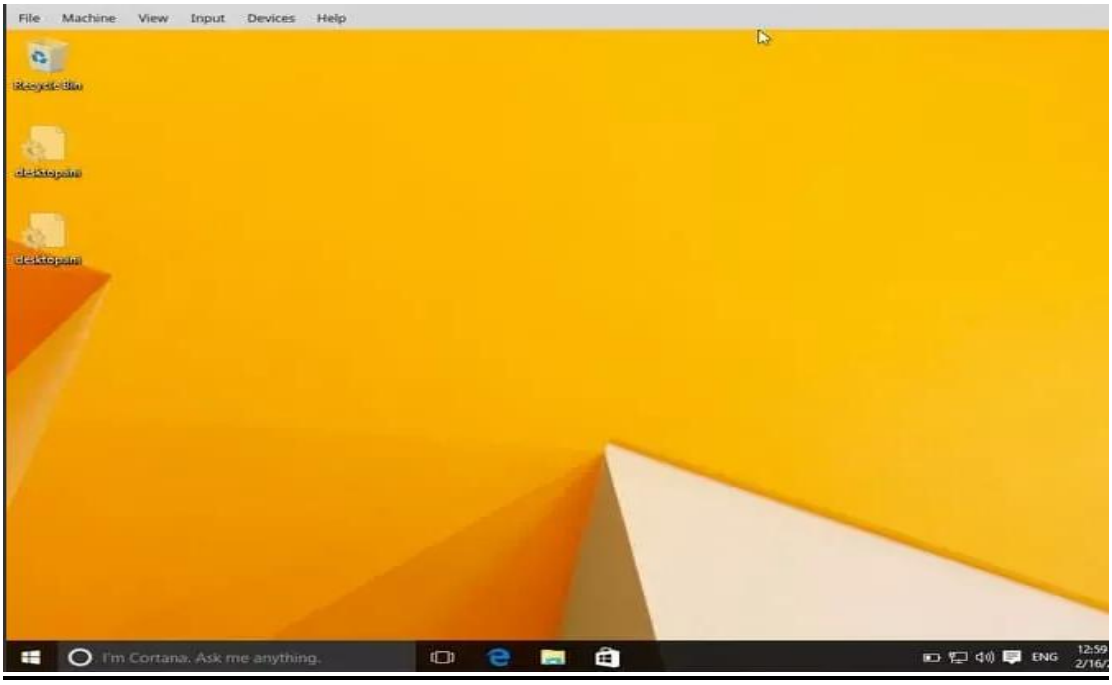
Step 6: Windows 11 Only (Important)

1. Windows 11 requires TPM + Secure Boot.
2. Enable TPM
3. **Settings → System**
4. Enable **TPM 2.0**
5. Go to **Processor**
    a. CPUs: **2 or more**
    b. Enable **PAE/NX**
6. **Enable EFI**
7. **Settings → System → Motherboard**
8. Check **Enable EFI**

Step 9: Start Windows Installation

1. Click **Start**
2. Windows installer will load
3. Choose:
    a. Language
    b. Keyboard
4. Click **Install Now**
5. When asked for product key:
    a. Click **I don't have a product key** (you can activate later)
6. Choose Windows edition
7. Select **Custom Install**
8. Select the virtual disk → **Next**

Windows will install automatically (10–30 minutes).



Conclusion

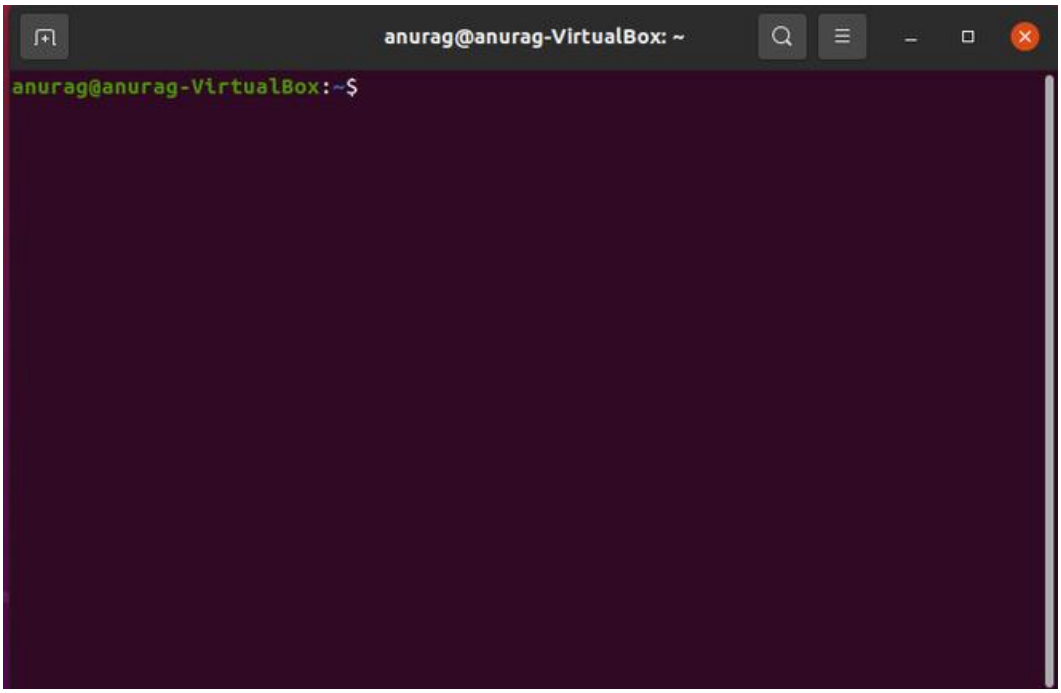
The experiment successfully demonstrated the installation of Windows on an Ubuntu system using Virtual Box. It showed how virtualization allows multiple operating systems to run on a single machine efficiently without affecting the host system.

EXPERIMENT – 5

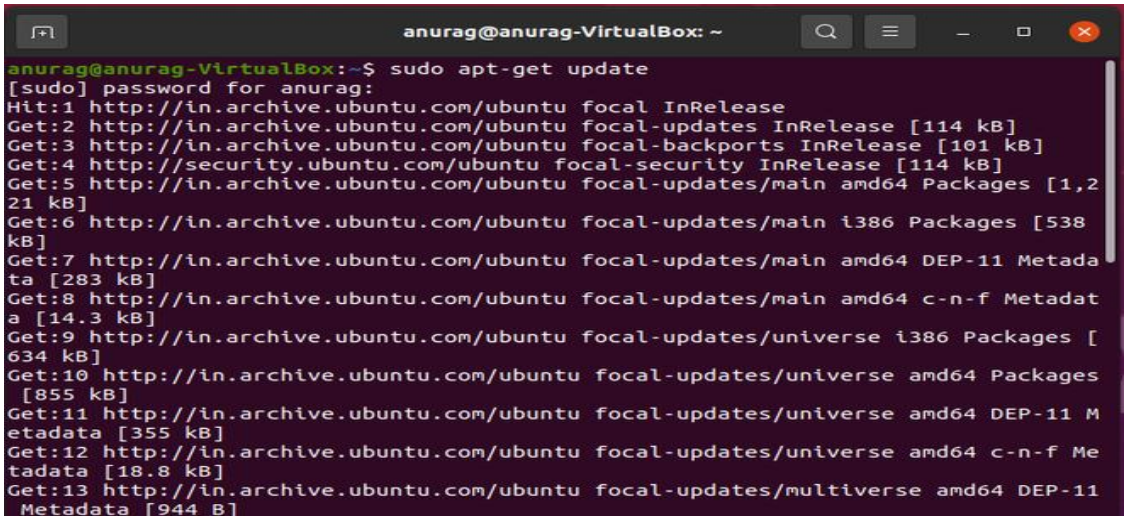
Aim : To install a C compiler and run a simple program in a virtual machine.

Procedure :

1. Open the **Terminal** in Ubuntu.



2. Update the package list using the command: **sudo apt update**



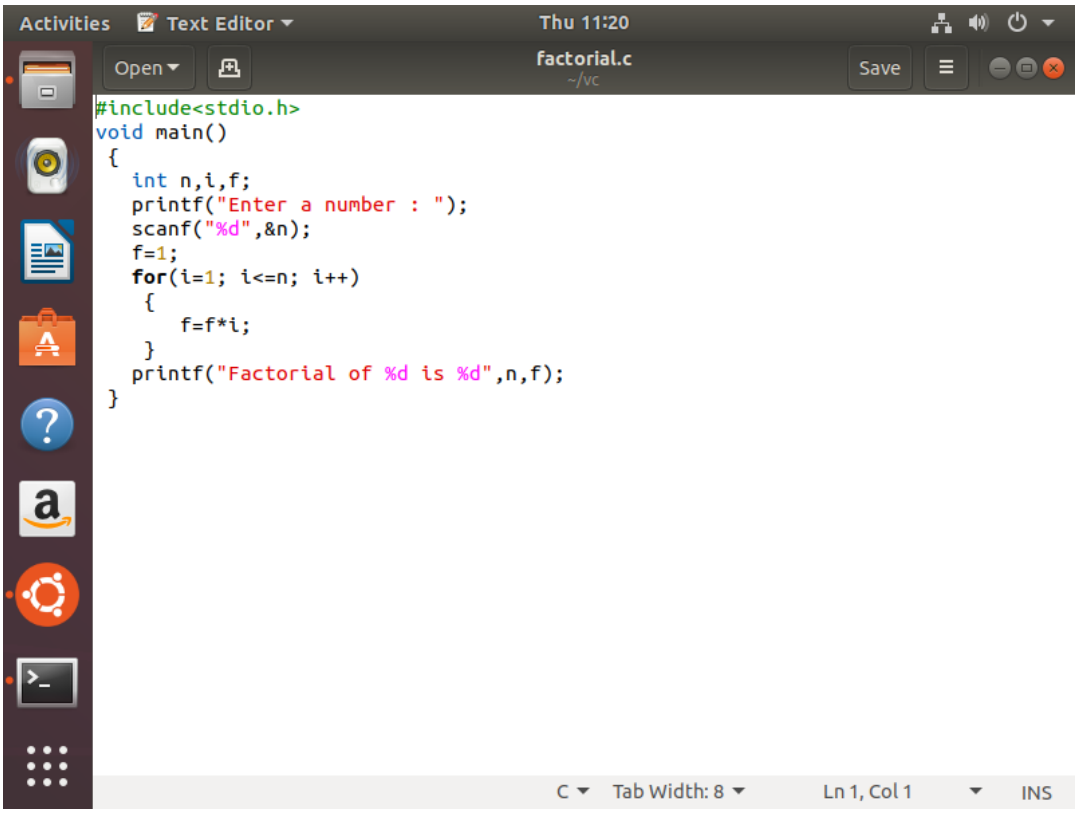
3. Install GCC compiler using the command: **sudo apt install gcc**

```
[1]+  Stopped                 rails s
lauraagal@lauraagal-VivoBook-S15-X510UF:~/Διπλωματική/login2$ sudo apt-get install gcc
[sudo] password for lauraagal:
Reading package lists... Done
Building dependency tree
Reading state information... Done
Some packages could not be installed. This may mean that you have
requested an impossible situation or if you are using the unstable
distribution that some required packages have not yet been created
or been moved out of Incoming.
The following information may help to resolve the situation:

The following packages have unmet dependencies.
 gcc : Depends: gcc-7 (>= 7.3.0-12~) but it is not going to be installed
E: Unable to correct problems, you have held broken packages.
lauraagal@lauraagal-VivoBook-S15-X510UF:~/Διπλωματική/login2$
```

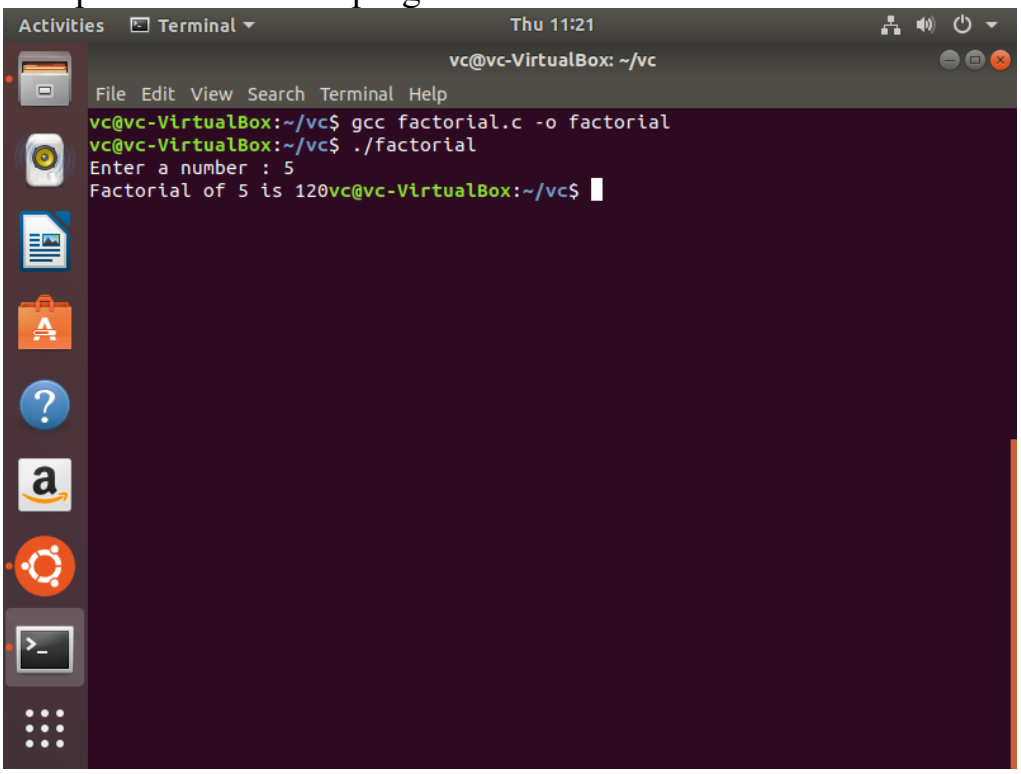
4. Press **Y** and **Enter** when prompted to confirm installation.
5. After installation, verify GCC by typing: **gcc –version**

```
anurag@anurag-VirtualBox:~$ gcc --version
gcc (Ubuntu 9.3.0-17ubuntu1~20.04) 9.3.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

6. Confirm that the GCC version details are displayed, indicating successful installation.
7. Write a C program to calculate the factorial of a number



8. Compile and run the C program



Conclusion

The GCC compiler was successfully installed on the Ubuntu system. The installation was verified using the version command, confirming that the system is now ready for compiling and executing C programs.

CYCLE - 2

EXPERIMENT – 6

Aim : Execute and comprehend fundamental networking commands (any 4 commands) within the Ubuntu virtual box environment. Describe how it contributes to diagnosing network connectivity problems, obtaining network-related data, or configuring network settings. Document your observations and explanations in a record

Commands :

1. Ifconfig

```
.
Unpacking net-tools (1.60+git20161116.90da8a0-1ubuntu1) ...
Processing triggers for man-db (2.8.3-2ubuntu0.1) ...
Setting up net-tools (1.60+git20161116.90da8a0-1ubuntu1) ...
vboxuser@ubuntu:~$ ifconfig
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.0.2.15  netmask 255.255.255.0  broadcast 10.0.2.255
        inet6 fe80::d3eb:5117:4fc6:aa37  prefixlen 64  scopeid 0x20<link>
        ether 08:00:27:0b:4d:38  txqueuelen 1000  (Ethernet)
        RX packets 562  bytes 635753 (635.7 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 339  bytes 42256 (42.2 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 192  bytes 17474 (17.4 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 192  bytes 17474 (17.4 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

vboxuser@ubuntu:~$
```

Observation:
Displays network interface details such as IP address, broadcast address, and packet statistics.

Explanation / Use:
This command is used to view and troubleshoot network interface configuration and detect packet transmission errors.

2. Ping

```
vboxuser@ubuntu:~$ ping google.com
PING google.com (142.251.43.46) 56(84) bytes of data.
64 bytes from pnmaaa-av-in-f14.1e100.net (142.251.43.46): icmp_seq=1 ttl=117 tim
e=24.4 ms
64 bytes from pnmaaa-av-in-f14.1e100.net (142.251.43.46): icmp_seq=2 ttl=117 tim
e=27.3 ms
64 bytes from pnmaaa-av-in-f14.1e100.net (142.251.43.46): icmp_seq=3 ttl=117 tim
e=24.5 ms
64 bytes from pnmaaa-av-in-f14.1e100.net (142.251.43.46): icmp_seq=4 ttl=117 tim
e=26.5 ms
64 bytes from pnmaaa-av-in-f14.1e100.net (142.251.43.46): icmp_seq=5 ttl=117 tim
e=33.3 ms
64 bytes from pnmaaa-av-in-f14.1e100.net (142.251.43.46): icmp_seq=7 ttl=117 tim
e=16.8 ms
64 bytes from pnmaaa-av-in-f14.1e100.net (142.251.43.46): icmp_seq=8 ttl=117 tim
e=23.8 ms
64 bytes from pnmaaa-av-in-f14.1e100.net (142.251.43.46): icmp_seq=9 ttl=117 tim
e=23.4 ms
```

Observation:
Replies received with time values indicate successful connectivity.

Explanation / Use:
`ping` checks network connectivity between the system and a remote host. It helps diagnose whether the system can access the internet or another device on the network.

3. NSLOOKUP

```
vboxuser@ubuntu:~$ nslookup google.com
Server:         127.0.0.53
Address:        127.0.0.53#53

Non-authoritative answer:
Name:   google.com
Address: 142.251.43.46
Name:   google.com
Address: 2404:6800:4007:834::200e
```

Observation

The command displays:

- DNS server address used for the query
- IP address(es) associated with the given domain name

Explanation / Use

nslookup is used to query DNS servers and resolve domain names into IP addresses. It helps in diagnosing DNS-related issues by verifying whether a domain name is correctly resolved. If a website is unreachable but the network is active, nslookup helps determine whether the problem is due to DNS failure or network connectivity issues.

4. Traceroute

```
vboxuser@ubuntu:~$ traceroute google.com
traceroute to google.com (142.250.183.174), 30 hops max, 60 byte packets
 1  _gateway (10.0.2.2)  1.766 ms  1.674 ms  1.540 ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *^C
vboxuser@ubuntu:~$ s
```

Observation

The command displays a list of intermediate routers (hops) between the source and destination, along with the time taken for each hop.

Explanation / Use

`traceroute` helps identify the route followed by packets across the network. It is useful for diagnosing network delays, packet loss, or routing problems by showing where the connection slows down or fails.

5. Netstat

```
ubuntu@ubuntu:~$ netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 ubuntu:59644            ubuntu-archive-mir:http TIME_WAIT
tcp        0      0 ubuntu:35760            ec2-52-0-238-118.:https ESTABLISHED
tcp        0      0 ubuntu:52826            dashboard.snapcra:https ESTABLISHED
tcp        0      0 ubuntu:42060            ubuntu-mirror-1.ps:http TIME_WAIT
tcp        0      0 ubuntu:54620            api.snapcraft.io:https  ESTABLISHED
tcp        0      0 ubuntu:53728            ubuntu-mirror-2.ps:http TIME_WAIT
Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags       Type       State         I-Node   Path
unix  21     [ ]         DGRAM                    13182    /run/systemd/journal/dev-log
unix  2      [ ]         DGRAM                    23691    /run/user/1000/systemd/notify
unix  2      [ ]         DGRAM                    20452    /run/user/121/systemd/notify
unix  3      [ ]         DGRAM                    12963    /run/systemd/notify
unix  2      [ ]         DGRAM                    12982    /run/systemd/journal/syslog
unix  9      [ ]         DGRAM                    12989    /run/systemd/journal/socket
unix  3      [ ]         STREAM     CONNECTED     26505    @/tmp/dbus-nQPHD9Mscf
unix  3      [ ]         STREAM     CONNECTED     25843
unix  3      [ ]         STREAM     CONNECTED     22480
unix  3      [ ]         STREAM     CONNECTED     18545
unix  3      [ ]         STREAM     CONNECTED     17724    /var/run/dbus/system_bus_socket
unix  3      [ ]         STREAM     CONNECTED     22566    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     22487
unix  3      [ ]         STREAM     CONNECTED     22256    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     25506
unix  3      [ ]         STREAM     CONNECTED     24578
unix  3      [ ]         STREAM     CONNECTED     22640
unix  3      [ ]         STREAM     CONNECTED     26065
unix  3      [ ]         STREAM     CONNECTED     25685    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     25001
unix  3      [ ]         STREAM     CONNECTED     22398    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     21622
unix  3      [ ]         STREAM     CONNECTED     26506
unix  3      [ ]         STREAM     CONNECTED     25868
unix  3      [ ]         STREAM     CONNECTED     22483    /run/systemd/journal/stdout
unix  3      [ ]         STREAM     CONNECTED     21061    @/tmp/dbus-MiD367xJLg
```

Observation

The command displays:

- Active network connections on the system
- Listening ports and services
- Protocol used (TCP or UDP)
- Local and foreign IP addresses with port numbers
- State of the connection (LISTEN, ESTABLISHED, etc.)

Explanation / Use

netstat is used to display network statistics and active connections on a system. It helps in identifying open ports, monitoring network traffic, and checking which services are running and listening for connections. This command is useful for troubleshooting network issues and detecting unauthorized or unexpected network activity.

Conclusion

The experiment successfully demonstrated the execution of fundamental networking commands in the Ubuntu VirtualBox environment. These commands provide essential information for diagnosing network connectivity issues, analyzing network configuration, and understanding routing behavior in a Linux system.

EXPERIMENT – 7

Aim : Show the virtual machine migration based on the certain condition from one node to the other in cloud and also implement autoscaling.

Theory :

Virtual Machine (VM) migration is the process of moving a virtual machine from one physical host (node) to another without causing downtime or service interruption. VM migration helps in load balancing, server maintenance, energy efficiency, and better resource utilization in cloud environments.

VM scaling is the ability to adjust computing resources such as CPU, RAM, and storage allocated to a virtual machine based on system demand. This ensures that applications receive adequate resources when demand increases and reduces resource usage when demand decreases, thereby optimizing performance and cost efficiency.

Autoscaling automatically increases or decreases the number of virtual machines depending on workload conditions such as CPU utilization, memory usage, or network traffic.

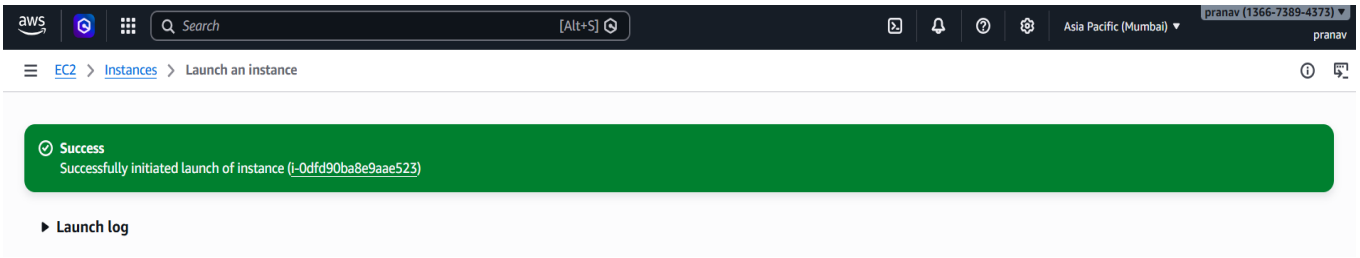
Algorithm for VM Migration

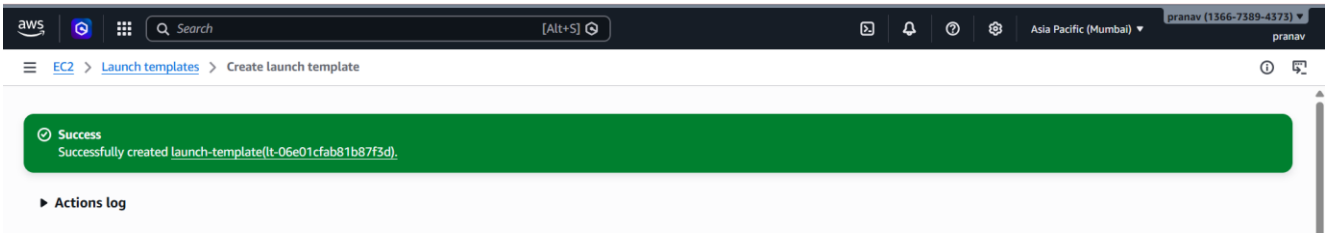
1. Start
2. Monitor the CPU utilization of all nodes in the cloud environment.
3. Check whether the CPU utilization exceeds the defined threshold value.
4. If CPU usage is greater than the threshold (e.g., 80%), select the VM from the overloaded node.
5. Identify another node with lower CPU utilization.
6. Migrate the selected VM to the identified node.
7. Continue monitoring the system resources.
8. Stop

Procedure for Auto Scaling:

Step 1:

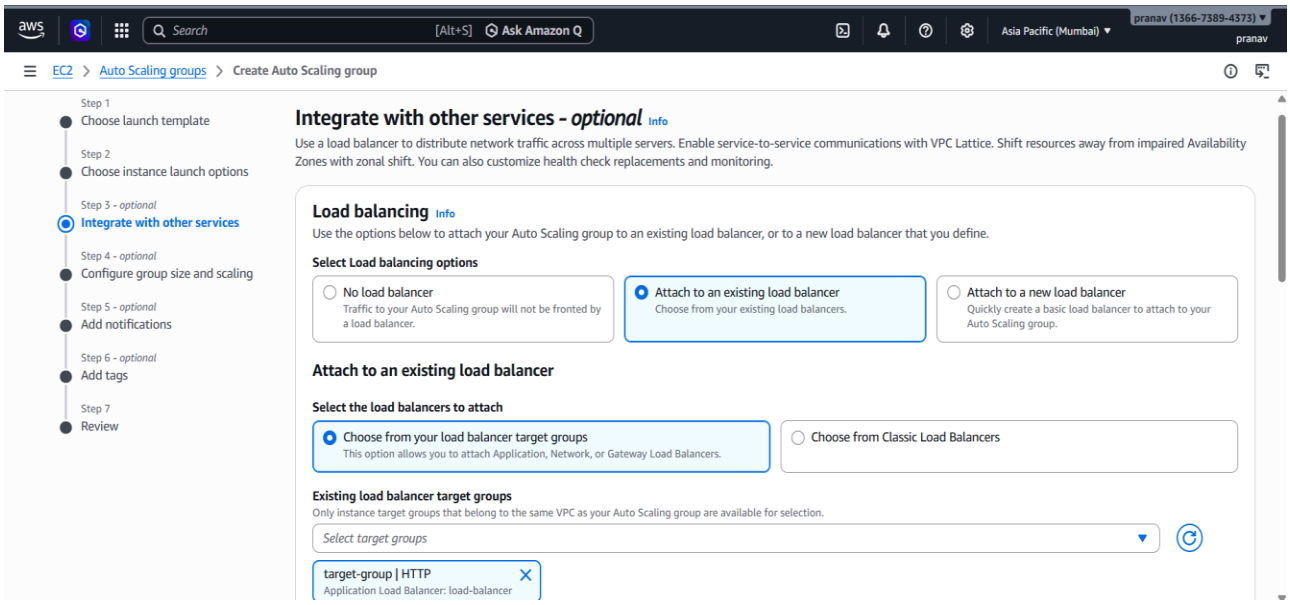
1) Go to EC2 → Launch Templates → Create launch template
2) Give template name
3) Application and OS Images -> Quick start → Ubuntu
4) Instance type → t3.micro
5) Key pair → select the same key used for EC2 instances
6) Security group → select the same security group used by your EC2 instances (the one allowing HTTP) (two in total)
7) Go to Advanced details → User data and paste:
    i) #!/bin/bash
    ii) apt update -y
    iii) apt install apache2 -y
    iv) systemctl start apache2
    v) systemctl enable apache2
    vi) echo "<h1>Server $(hostname)</h1>" > /var/www/html/index.html
8) Click Create launch template





Step 2:Create Auto Scaling Group

1) Go to **EC2 → Auto Scaling Groups → Create**
2) Give group name
3) Select the launch template created above
4) Select two availability zones
5) Click next
6) **Load balancing section**
   a) Attach to existing load balancer
   b) Choose from your balancer target groups
   c) Select your load balancer target group
7) Select **VPC** (same as EC2 + Load Balancer)
8) Select **at least two subnets** (same AZs as Load Balancer)
9) Click **Next**
10) **Health check section**
11) Turn on Elastic Load Balancing health checks
12) Click next

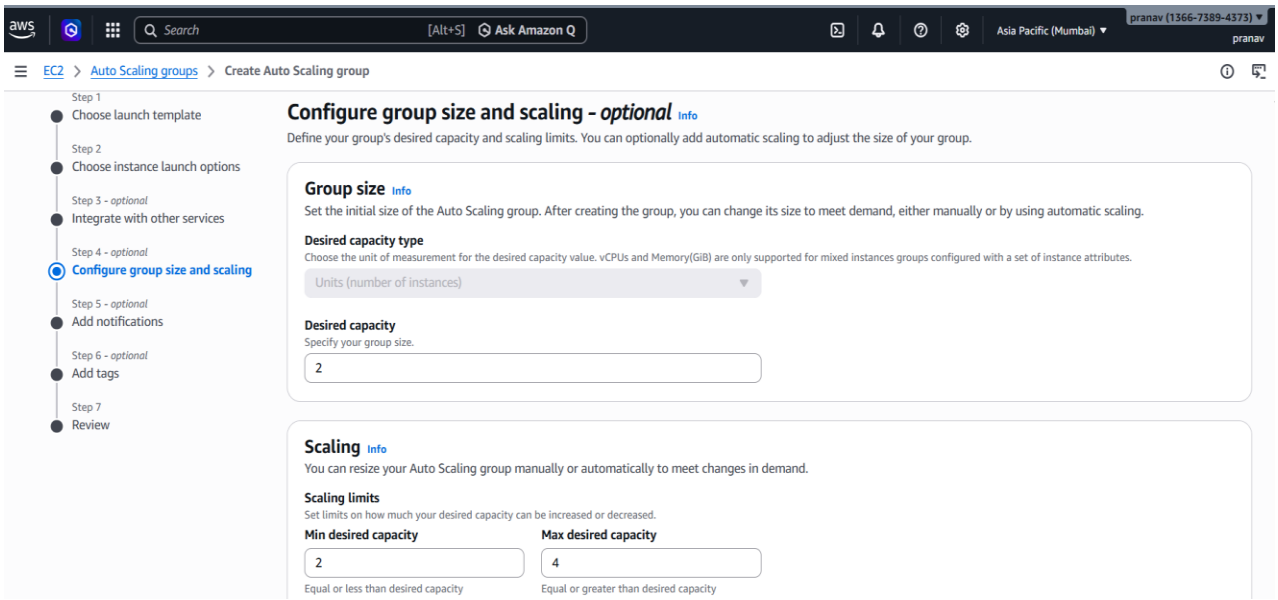


Step 3: Group Size

Set:

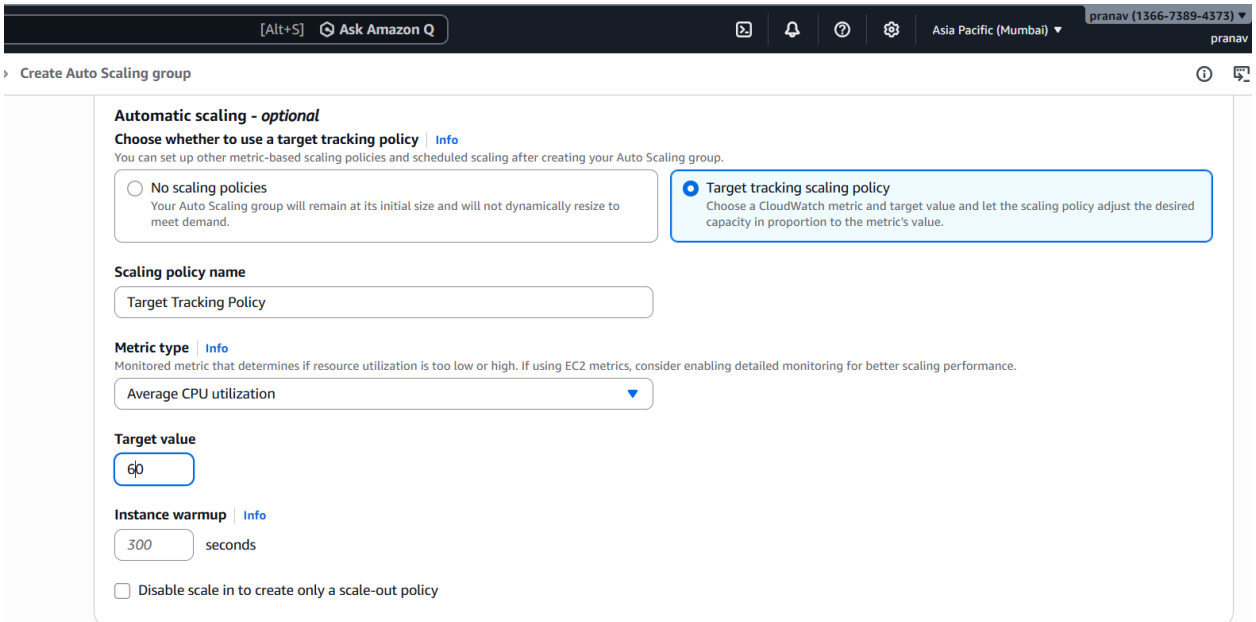
1) Desired capacity: **2**
2) Minimum capacity: **2**

3) Maximum capacity: **4**
4) Click **Next**



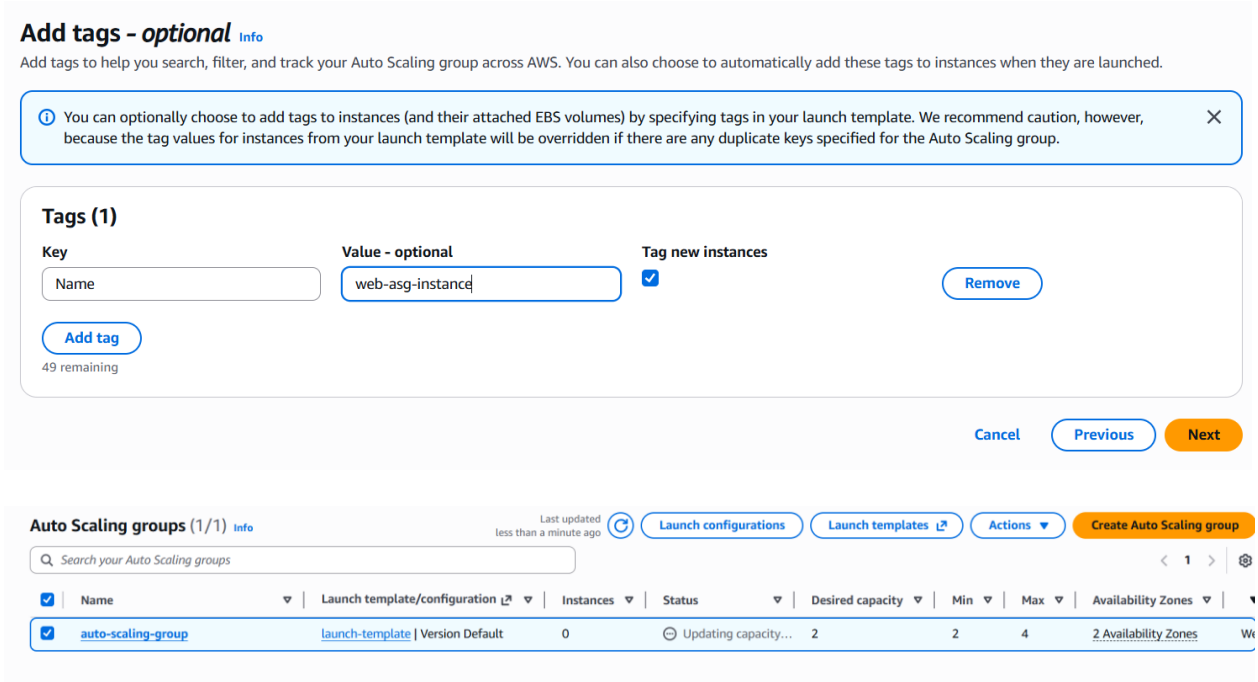
Step 4: Scaling Policy

1) Choose **Target tracking scaling policy**
2) Metric type: **Average CPU utilization**
3) Target value: **60**
4) Instance warmup: **300 seconds**
5) Click **Next → Next**



Step 5: Tags

1) Add:
2) Key: **Name**
3) Value: **web-asg-instance**
4) Click **Next → Create Auto Scaling Group**



Step 6: Test Auto Scaling

1) Connect to one running instance:

sudo apt install stress -y

stress --cpu 2 --timeout 300

2)After a few minutes:

i. A new instance should appear in EC2
ii. It will automatically register in Target Group
iii. Load Balancer will start sending traffic

Step 7: Auto Replacement Test

1. If you terminate one instance manually:
2. Auto Scaling Group automatically launches a new instance
3. Registers it to Target Group
4. Load Balancer uses it

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
server1	i-0dfd90ba8e9aae523	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a	ec2-13-127-
web-asg-insta...	i-03b0a731b594e6b11	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a	ec2-13-232-
server2	i-08daec067d088f355	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1a	ec2-13-233-
web-asg-insta...	i-00678b6cda0ce1691	Running	t3.micro	3/3 checks passed	View alarms +	ap-south-1c	ec2-13-200-

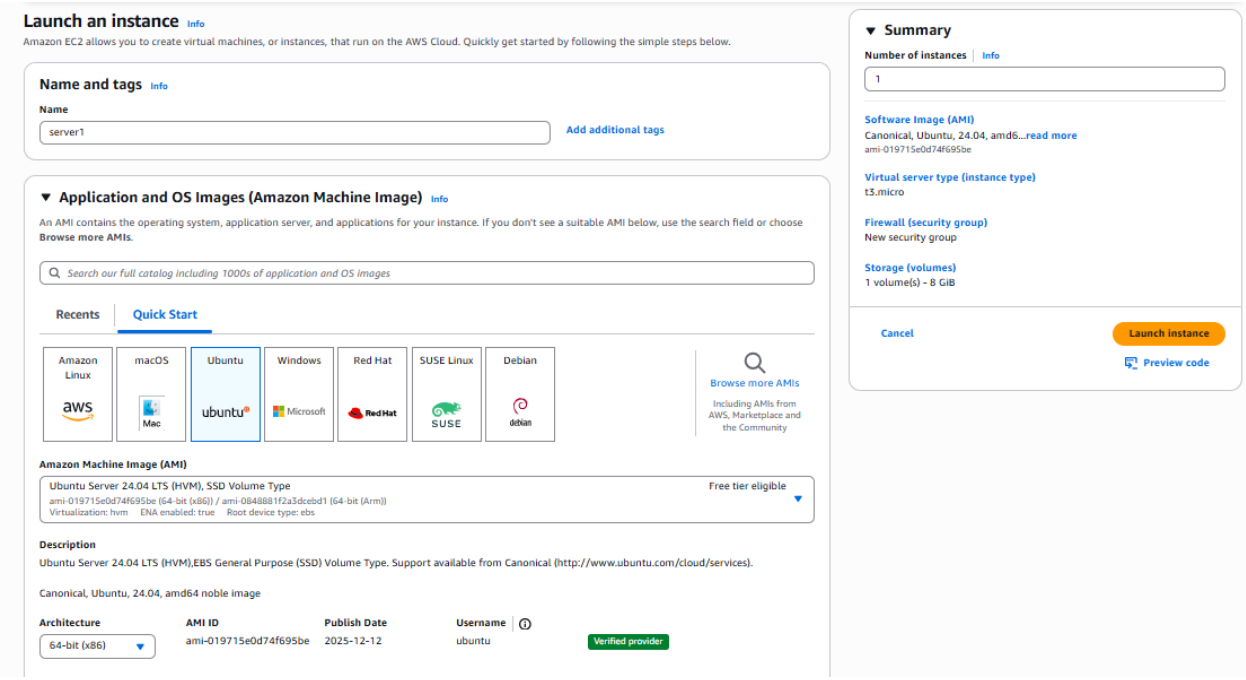
Result:
The Auto Scaling Group successfully launched and managed EC2 instances. When system load increased, new instances were automatically created and registered with the load balancer. Thus, VM migration and autoscaling in the cloud environment were successfully demonstrated.

CYCLE – 3

EXPERIMENT – 8

Aim : Create a load-balance VM cluster using AWS cloud service

1. First create two instance and make sure that allow http is turned on while creating an instance.



Name	Instance ID	Instance state	Instance type	Status check
server1	i-0707b7fbd7836b21b	Running	t3.micro	3/3 checks passed
server2	i-08097e22e516be87d	Running	t3.micro	3/3 checks passed

2. Install webserver(apache) in both.

sudo apt update -y

sudo apt install apache2 -y

sudo systemctl start apache2

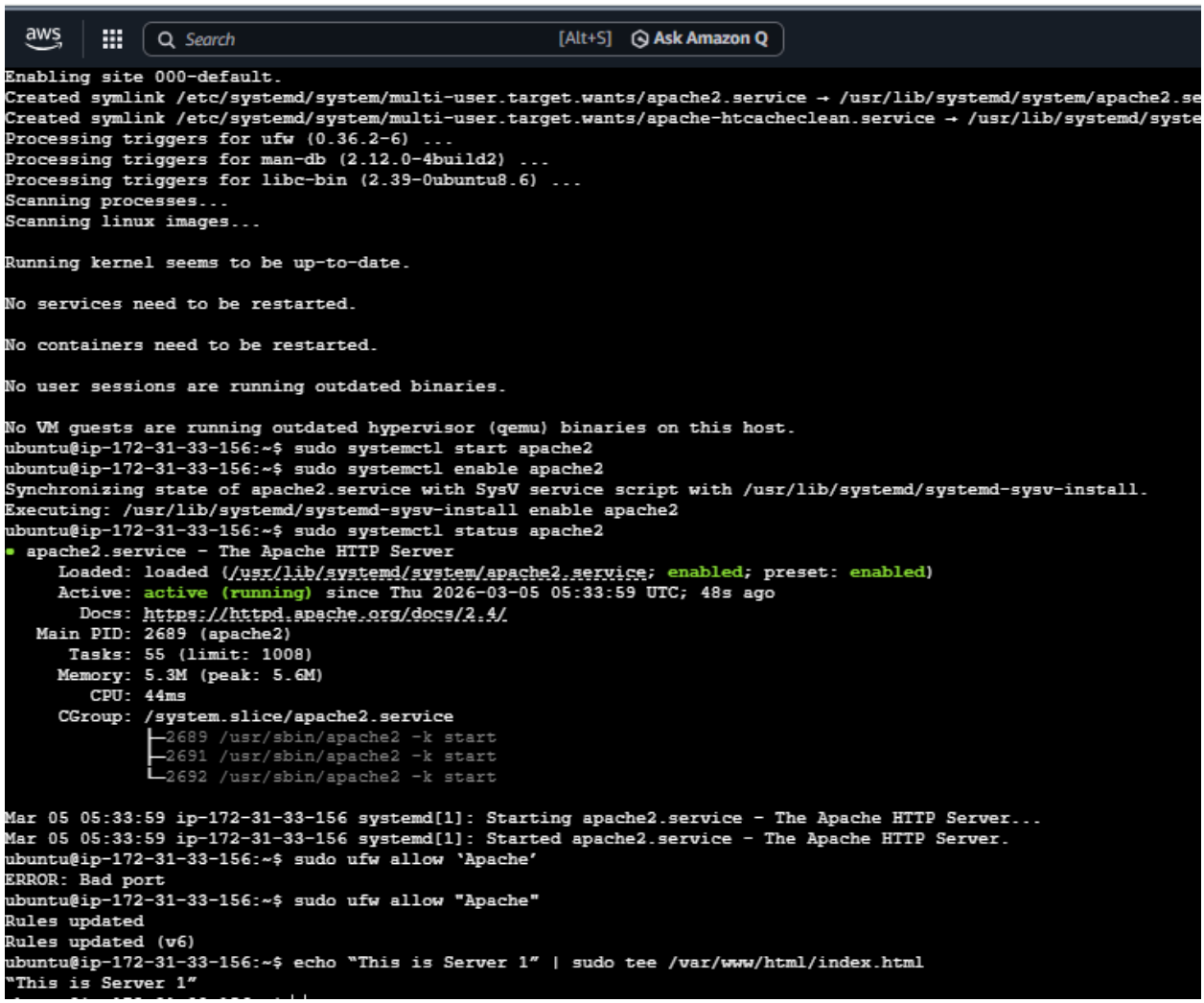
sudo systemctl enable apache2

sudo systemctl status apache2

sudo ufw allow ‘Apache’

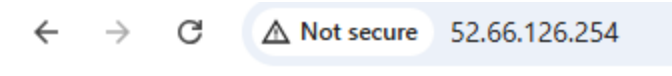
echo “This is Server 1” | sudo tee /var/www/html/index.html

echo “This is Server 2” | sudo tee /var/www/html/index.html

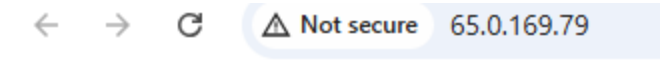


```
aws   Search [Alt+S] Ask Amazon Q
Enabling site 000-default.
Created symlink /etc/systemd/system/multi-user.target.wants/apache2.service → /usr/lib/systemd/system/apache2.se
Created symlink /etc/systemd/system/multi-user.target.wants/apache-htcacheclean.service → /usr/lib/systemd/syste
Processing triggers for ufw (0.36.2-6) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.6) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-33-156:~$ sudo systemctl start apache2
ubuntu@ip-172-31-33-156:~$ sudo systemctl enable apache2
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2
ubuntu@ip-172-31-33-156:~$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-03-05 05:33:59 UTC; 48s ago
       Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 2689 (apache2)
      Tasks: 55 (limit: 1008)
     Memory: 5.3M (peak: 5.6M)
        CPU: 44ms
     CGroup: /system.slice/apache2.service
             ├─2689 /usr/sbin/apache2 -k start
             ├─2691 /usr/sbin/apache2 -k start
             └─2692 /usr/sbin/apache2 -k start

Mar 05 05:33:59 ip-172-31-33-156 systemd[1]: Starting apache2.service - The Apache HTTP Server...
Mar 05 05:33:59 ip-172-31-33-156 systemd[1]: Started apache2.service - The Apache HTTP Server.
ubuntu@ip-172-31-33-156:~$ sudo ufw allow ‘Apache’
ERROR: Bad port
ubuntu@ip-172-31-33-156:~$ sudo ufw allow "Apache"
Rules updated
Rules updated (v6)
ubuntu@ip-172-31-33-156:~$ echo “This is Server 1” | sudo tee /var/www/html/index.html
“This is Server 1”
```

3) Then open browser and type http:// . Do for both server 1 and server2.



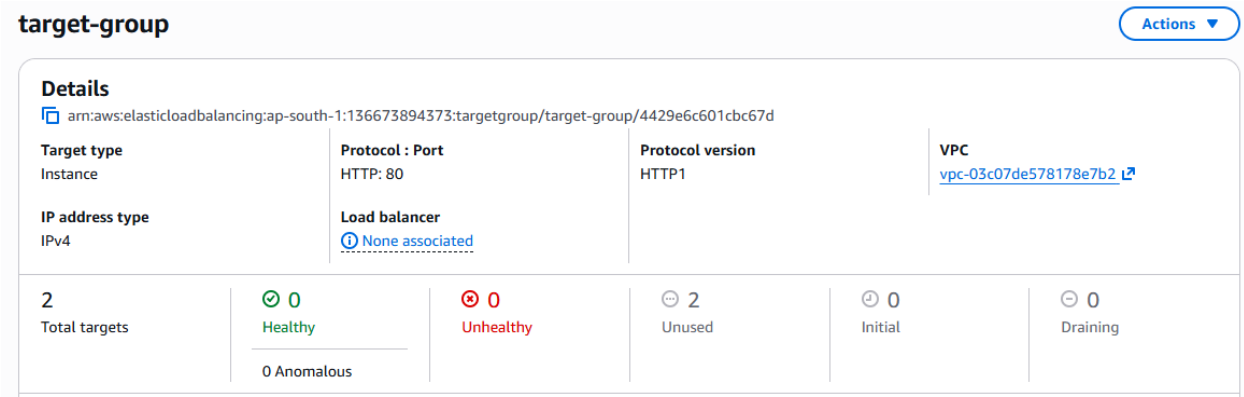
This is Server 1



This is Server 2

6. Now create a new target group

1. Go to EC2 → Target Groups → Create target group
2. Target type: Instances
3. Protocol: HTTP
4. Port: 80
5. VPC: same as EC2 instances
6. Health check path: /
7. Click Create
8. Now register instances:
    1. Open the created target group
    2. Go to Targets tab
    3. Click Register targets
    4. Select both instances
    5. Click Include as pending
    6. Click Register pending targets



5. Create Application Load Balancer.

1. Go to EC2 → Load Balancers → Create

2. Choose Application Load Balancer

3. Configure:

4. Scheme: Internet-facing

5. Listener: HTTP : 80

6. Availability Zones: select at least 2 AZs

7. Security group: click Create new security group

    1. (Inbound) HTTP 80 → Anywhere IPv4

    2. Create security group.

8. Now switch to creating application load balancer page

9. Listener routing: 1. Forward to → your Target Group

10. Click Create Load Balancer

    Wait until:

    1. State = Active

    2. Targets = Healthy

Create security group Info

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name Info

lbsecuritygroup

Name cannot be edited after creation.

Description Info

Allows SSH access to developers

VPC Info

vpc-03c07de578178e7b2

Inbound rules Info

Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
HTTP	TCP	80	Anywh... 0.0.0.0/0		Delete

Add rule

Create Application Load Balancer Info

The Application Load Balancer distributes incoming HTTP and HTTPS traffic across multiple targets such as Amazon EC2 instances, microservices, and containers, based on request attributes. When the load balancer receives a connection request, it evaluates the listener rules in priority order to determine which rule to apply, and if applicable, it selects a target from the target group for the rule action.

▶ How Application Load Balancers work

Basic configuration

Load balancer name

Name must be unique within your AWS account and can't be changed after the load balancer is created.

loadbalancer

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme | Info

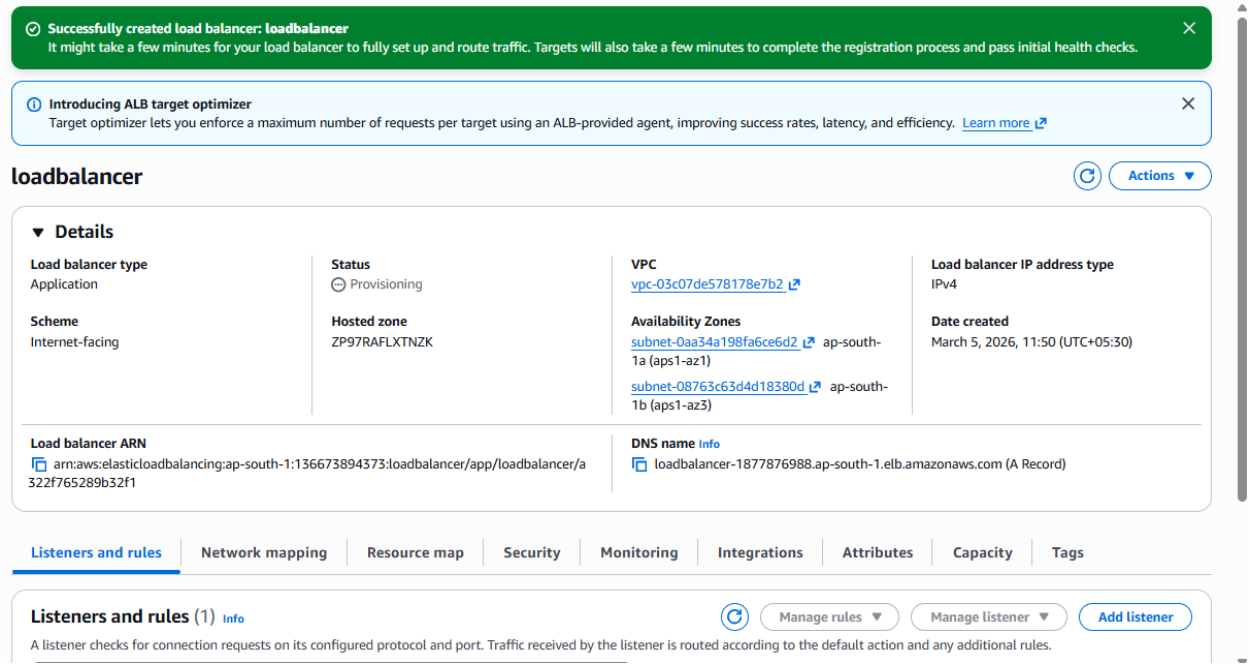
Scheme can't be changed after the load balancer is created.

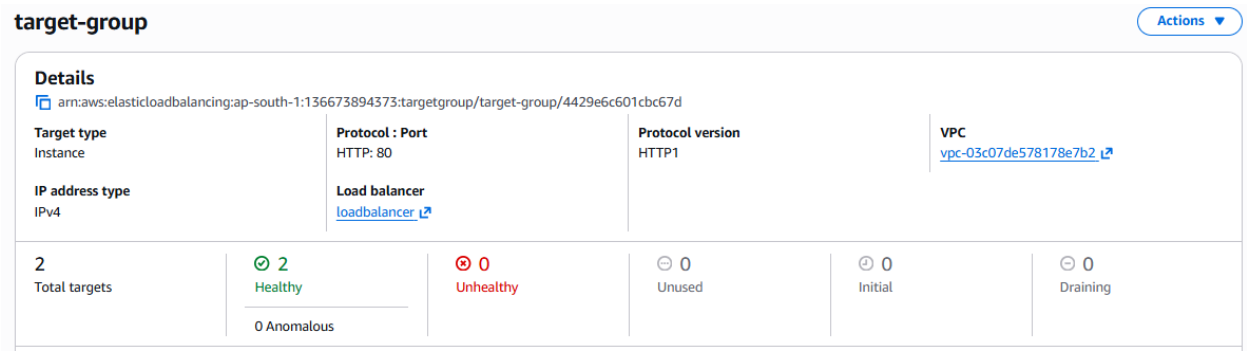
- (●) Internet-facing
  - Serves internet-facing traffic.
  - Has public IP addresses.
  - DNS name resolves to public IPs.
  - Requires a public subnet.
- ( ) Internal
  - Serves internal traffic.
  - Has private IP addresses.
  - DNS name resolves to private IPs.
  - Compatible with the **IPv4** and **Dualstack** IP address types.

Load balancer IP address type | Info

Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

- (●) IPv4
  Includes only IPv4 addresses.
- ( ) Dualstack
  Includes IPv4 and IPv6 addresses.
- ( ) Dualstack without public IPv4
  Includes a public IPv6 address, and private IPv4 and IPv6 addresses. Compatible with **internet-facing** load balancers only.





7. Allow traffic only from Load Balancer (Important — do after healthy)

   1. Now modify the EC2 instances security group:

   2. Remove rule: (inbound) HTTP → Anywhere

   3. Add rule:

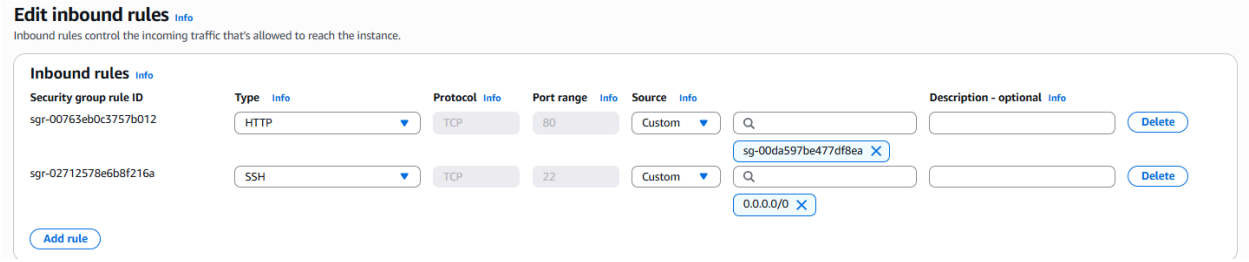
   4. Type: HTTP

   5. Source: Load Balancer Security Group

   6. Do this for both instances.

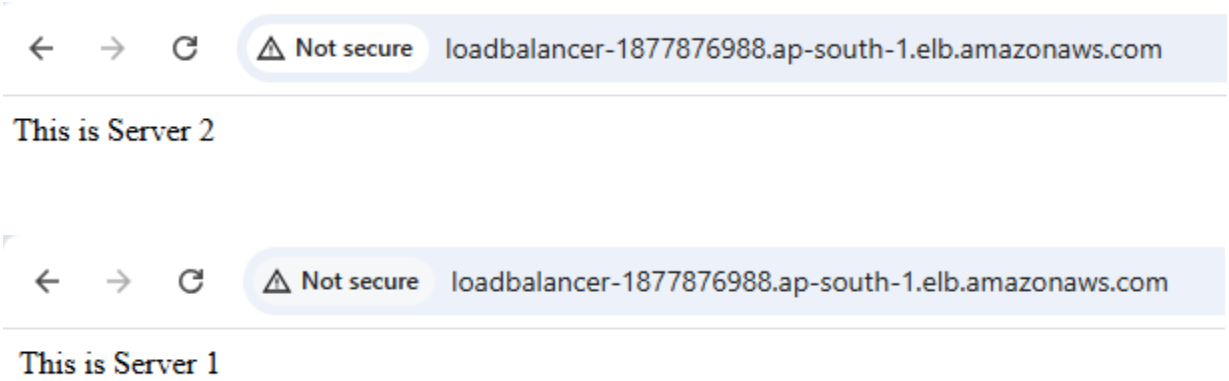
   7. Now instances accept traffic only from ALB.

   8. Now go to load balancer page.



8. Test Load Balancing

- Go to EC2 → Load Balancers
- Select your load balancer
- Copy DNS name
- Paste in browser Refresh repeatedly.

You should see:

Server 1 → Server 2 → Server 1 → Server 2